

<https://rupakm.github.io/leslie> <https://github.com/rupakm/leslie> <https://rupakm.github.io/leslie/docs>

Verifying ABA with Leslie

Gaspard Reghem

June 18, 2026

Abstract

Our main objective is the verification of an implementation of Asynchronous Byzantine Agreement. The proof of correctness consist in building a simulation relation between the implementation and an implementation running in an idealised setting. The simulation ensures that the correctness of the idealised implementation implies the correctness of the original one. However, this implementation is complex, thus, building the simulation in one go is intractable. Fortunately, the implementation is designed as the interlocking of multiple sub-protocols (GBCA, WCC, RSD, Gather, AVSS, BRB). This architecture allows us to decompose the construction of the overall simulation into smaller steps. Each implementation of sub-protocols will be simulated by an idealised version. Thus, when building the simulation for a protocol using sub-protocols, the sub-implementations will be replaced by their idealised versions; making the construction easier.

0.1 Preliminaries

0.1.1 Traditionnal Framework

Definition 1 (Probabilistic Labelled Transition System). PLTS

A probabilistic labelled transition system (PLTS) is a tuple $\mathcal{P} := (Q, q_\star, L, T)$ where Q is a (countable) set of states, $q_\star \in Q$ is a set of initial states, L is a (countable) set of labels and $T \subseteq Q \times (L \cup \{\tau\}) \times \mathcal{D}(Q)$ is a set of probabilistic transitions.

A partial execution of $\mathcal{P}$ is sequence $q_0.l_0.q_1.l_1 \cdots \in (Q.L)^*.Q \cup (Q.L)^\omega$ such that, for all $n \in \mathbb{N}$, there exists $\mu_n \in \mathcal{D}(Q)$ such that $(q_n, l_n, \mu_n) \in T$ and $q_{n+1} \in \bar{\mu}_n$. The set of finite executions is denoted $\mathcal{E}^*$. A partial execution is called an execution if $q_0 = q_\star$. The trace of a partial execution is its projection to L . The set of traces of $\mathcal{P}$ is denoted $\mathcal{T}$. A scheduler s is function from $\mathcal{E}^*$ to $\mathcal{D}(T \cup \{\perp\})$ such that, for all $e \in \mathcal{E}^*$ and $t \in s(e)$, $t = \perp$ or $src(t) = last(e)$. The set of schedulers is denoted as Sch .

Definition 2 (Partial Probabilistic Execution). Partial Probabilistic Execution

Given $\mu \in \mathcal{D}(Q)$ and $s \in Sch$, one can define a partial probabilistic execution induced by (μ, s) as the function $\mathbf{e}_{\mu,s}$ from $\mathcal{E}^*$ to $[0, 1]$ such that if $e \in Q$ then $\mathbf{e}_{\mu,s} = \mu(e)$; else, $e = e'.l.q$ and $\mathbf{e}_{\mu,s}(e) = \mathbf{e}_{\mu,s}(e') \cdot \sum_{t \in T | lbl(t)=l} s(e')(t) \cdot out(t)(q)$.

A partial probabilistic execution is called a probabilistic execution if it is induced by a pair (μ, s) such that $\mu = \delta_{q_\star}$. Partial probabilistic execution induce a measure on the σ -field induced by cones of partial executions and the trace function is measurable from the σ -field induced by cones of partial executions to the one induced by cones of traces. Thus, $\mathbf{t}_{\mu,s} = tr^{-1} \circ \mathbf{e}_{\mu,s}$ is well-defined and called a trace distribution. The set of achievable trace distribution by $\mathcal{P}$ is defined as those induced by a probabilistic execution of $\mathcal{P}$ and denoted $tdist(\mathcal{P})$. The trace distribution pre-order $\mathcal{P} \subseteq^t \mathcal{P}'$ is defined by $tdist(\mathcal{P}) \subseteq tdist(\mathcal{P}')$.

Definition 3 (Parallel Composition of PLTS). Parallel Composition

Given $\mathcal{P}$ and $\mathcal{P}'$ two PLTSs and $S \subseteq L \cup L'$, the composed PLTS $\mathcal{P} \parallel_S \mathcal{P}'$ is defined as $(Q \times Q', (q_\star, q'_\star), L \cup L', T \parallel_S T')$ where $((q, q'), l, \mu'') \in T \parallel_S T'$ if and only if

- $l \in S$ and there exist $(q, l, \mu) \in T$ and $(q', l, \mu') \in T'$ such that $\mu \otimes \mu' = \mu''$; or
- $l \notin S$ and there exists $(q, l, \mu) \in T$ such that $\mu \otimes \delta_{q'} = \mu''$; or
- $l \notin S$ and there exists $(q', l, \mu') \in T'$ such that $\delta_q \otimes \mu' = \mu''$.

The trace distribution pre-order is not stable by $\parallel_S$: $\mathcal{P} \subseteq^t \mathcal{P}'$ does not imply $\mathcal{P} \parallel_S \mathcal{P}'' \subseteq^t \mathcal{P}' \parallel_S \mathcal{P}''$ [?]. Thus, the largest pre-order included in $\subseteq^t$ stable by $\parallel_S$ is called the trace distribution pre-congruence and is denoted $\sqsubseteq^t$.

Let $R \subseteq A \times \mathcal{D}(B)$, the probabilistic lifting of R , denoted $\mathcal{D}(R)$, is defined as $(\mu_A, \mu_B) \in \mathcal{D}(R)$ if and only if there exists $w : R \rightarrow [0, 1]$ such that $\mu_A =$

$a \mapsto \sum_{\mu \in \mathcal{D}(B)} w(a, \mu)$ and $\mu_B = \sum_{(a, \mu) \in R} w(a, \mu) \cdot \mu$. If $s \in Sch$ terminates almost-surely from μ , (i.e. $\sum_{e \in \mathcal{E}^*} \mathbf{e}_{\mu, s}(e) \cdot s(e)(\perp) = 1$) then $\mathbf{q}_{\mu, s} = last^{-1} \circ \mathbf{e}_{\mu, s}$ is a well-defined probability distribution over states. We denote $\mu l \mu'$ if and only if there exists $s \in Sch$ such that $\delta_l = \mathbf{t}_{\mu, s}$ and $\mu' = \mathbf{q}_{\mu, s}$.

Definition 4 (Probabilistic Forward Simulation). Probabilistic Forward Simulation

Given $\mathcal{P}$ and $\mathcal{P}'$ two PLTSs, a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times \mathcal{D}(Q')$ such that $(q_*, \delta_{q'_*}) \in \mathcal{S}$ and, for all $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$; there exists $\mu' l \mu''$ such that $(\mu, \mu'') \in \mathcal{D}(\mathcal{S})$.

Theorem 1 (from [?]). *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. The proof is non-trivial and detailed in Segala's PhD. $\square$

In our setting, the most interesting implication of Theorem ?? is the soundness. We will see that the definition of $\sqsubseteq^t$ has some limitations, notably with respect to liveness. To solve the issue, we will introduce a stronger notion of trace distribution and strengthen probabilistic forward simulation alongside. However, the soundness proof of Segala [?] works at the level of $\mathbf{e}_{\mu, s}$, not s . This is problematic because our strengthening consists in constraining schedulers, thus, we need to adapt the proof. If $\mathcal{S}$ is a probabilistic forward simulation then it induces a choice function $\mathcal{C} : \mathcal{S} \times T \rightarrow Sch \times (\mathcal{S} \rightarrow [0, 1])$ which maps any $(q, \mu') \in \mathcal{S}$ and $(q, l, \mu) \in T$ to the scheduler that induces $\mu' l \mu''$ and the witness w of $(\mu, \mu'') \in \mathcal{S}$.

Once the choice function $\mathcal{C}$ is fixed, Algorithm ?? is a sequential probabilistic program, thus, it can be modelled as a deterministic PLTS. Of course, the validity of the different instructions of the algorithm should be proven properly, but it is not the point. The variable e encodes an execution of $\mathcal{P}$ and acc an execution of $\mathcal{P}'$ that simulates it. In order to extract a scheduler of $\mathcal{P}'$ from Algorithm ??, we need to identify instructions deciding which transitions to execute in $\mathcal{P}'$. The most obvious one is $t' \leftarrow s_t(e')$ when $t' \neq \perp$, then there is $t \leftarrow s(e)$ when $t = \perp$. The case where $t' = \perp$ have to be discarded because another t' will then be picked without updating acc . When $t = \perp$ then the algorithm stops and so the scheduler on $\mathcal{P}'$ would also return $\perp$. After each update of acc , one of the three following events will happen: (i) t is set to $\perp$; (ii) t' is set to a non- $\perp$ value and (iii) t' is set to $\perp$ indefinitely. Cases (i) and (iii) correspond to scheduling $\perp$ and case (ii) corresponds to scheduling t' . Thus, the key values to compute are: $reach(e')$ the probability of reaching a configuration where $acc = e'$ and $sch(e', t)$ the probability of reaching a configuration where $acc = e$ and $t' = t$. Knowing these, one can define a scheduler on $\mathcal{P}'$ as

$$s' : e' \mapsto \begin{cases} t' \mapsto \frac{sch(e', t')}{reach(e')} \\ \perp \mapsto 1 - \frac{\sum_{t' \in T'} sch(e', t')}{reach(e')} \end{cases}$$

I am quite convinced that $\mathbf{t}_{q_*, s} = \mathbf{t}_{q'_*, s'}$.

0.1.2 Non-probabilistic Systems

Non-probabilistic systems can be considered as special instances of PLTSs where all transitions lead to a Dirac distribution. By identifying Dirac distributions to the unique state in their support, we can assume that non-probabilistic systems are modelled by LTS which leads to some simplifications. In particular, probabilistic forward simulation is equivalent to a simpler simulation relation. On LTS, we denote qlq' if there exists a partial execution from q to q' whose trace is l .

Definition 5 (Weak Simulation). Weak Simulation

Given $\mathcal{P}$ and $\mathcal{P}'$ two LTSs, a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ is a relation $\mathcal{S} \subseteq Q \times Q'$ such that $(q_*, q'_*) \in \mathcal{S}$ and, for all $(q, q') \in \mathcal{S}$ and $(q, l, p) \in T$; there exists $q'lp'$ such that $(p, p') \in \mathcal{S}$.

Theorem 2. *Given two LTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{P} \sqsubseteq^t \mathcal{P}'$ if and only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. According to Theorem ??, it suffices to prove that there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if there exists a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$. For one direction, it suffices to note that any weak simulation is a probabilistic weak simulation. For the other direction, if $\mathcal{S}$ is a probabilistic forward simulation of $\mathcal{P}$ by $\mathcal{P}'$ then $\{(q, q') \mid \exists (q, \mu) \in \mathcal{S}, q' \in \bar{\mu}\}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$. $\square$

There is a natural injection of LTSs to PLTSs, but also a natural projection of PLTSs onto LTSs. It consists in transforming all the probabilistic non-determinism into classical non-determinism. The projection of a PLTS $\mathcal{P}$, denoted $\rho(\mathcal{P})$, is the LTS whose transition are defined by $\{(q, l, q') \mid \exists \mu \in \mathcal{D}(Q), (q, l, \mu) \in T \wedge q' \in \bar{\mu}\}$. ρ is trivially the identity on LTSs and ρ preserves the set of executions and so the set of traces. As a result, the notion of weak simulation can be lifted to PLTSs: $\mathcal{S}$ is a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$ if and only if it is a weak simulation of $\rho(\mathcal{P})$ by $\rho(\mathcal{P}')$. Since ρ is a projection on LTSs, this is consistent with the previous definition.

Theorem 3. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T} \subseteq \mathcal{T}'$ only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. The proof is folklore. For each execution of $\mathcal{P}$, an execution of $\mathcal{P}'$ can inductively be built such that they share the same trace. $\square$

0.1.3 Preserving Liveness

Our current definitions are not adapted to speak about liveness. Indeed, $\mathcal{E}$ always contain the empty execution q_* , thus, $\mathcal{T}$ always contain the empty trace τ . As a result, no liveness property can be satisfied against such a set of executions. However, intuition tells us that such executions should be discarded when evaluating properties because we implicitly make a progress assumption: "if a transition is enabled then a transition should be performed".

To take into account this observation, we say that an execution is maximal if it is infinite or its last state is a deadlock. Similarly, we say that a trace is maximal if it is the trace of a maximal execution. According to the progress assumption, properties should be evaluated against the set of maximal traces. Therefore, we must develop simulation relations that preserve the set of maximal traces. The set of maximal traces is denoted $\mathcal{T}^{\max}$.

However, weak simulation does not preserve the set of maximal traces. First, there exists a trivial weak simulation of a deadlock by any LTS. Then, since τ -transitions can be elided, an infinite execution whose trace is τ , called a divergence, can be simulated by stuttering. We call a weak divergence any execution whose trace is τ and which is infinite or ends by a deadlock. We also say that a state weakly diverges if a weak divergence starts from it.

Definition 6 (Maximal Weak Simulation). Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be maximal if, for all $(q, q') \in \mathcal{S}$, if q weakly diverges then q' weakly diverges.

Theorem 4. *Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, $\mathcal{T}^{\max} \subseteq \mathcal{T}'^{\max}$ only if there exists a weak simulation of $\mathcal{P}$ by $\mathcal{P}'$.*

Proof. It suffices to adapt the proof of Theorem ?? . When building an execution e' of $\mathcal{P}'$ that simulates a maximal execution e of $\mathcal{P}$, it suffices to check that the additional requirement on the simulation ensures that e' is maximal. If e contains infinitely many non- τ labels then the construction will naturally yield an infinite execution. Otherwise, after simulating the smallest finite prefix that contains all non- τ labels, it suffices to trigger the weak divergence preservation. $\square$

However, this solution is not satisfactory because the additional requirement asks to test for the existence of weak divergences in $\mathcal{P}$. In practice, $\mathcal{P}$ will be a PLTS encoding a real implementation whose size and complexity will make this search difficult. To get around this issue, simulations should rely on a test that can be performed locally (e.g. checking if a transition is enabled), not globally (e.g. checking whether a divergence exists). We allow a global test of $\mathcal{P}'$ because it will encode a minimal PLTS satisfying a set of properties. We say that a transition is elided by a weak simulation if the choice function returns

Definition 7 (Progressive Weak Simulation). Given two PLTSs $\mathcal{P}$ and $\mathcal{P}'$, a weak simulation $\mathcal{S}$ of $\mathcal{P}$ by $\mathcal{P}'$ is said to be progressive if

- for all $(q, q') \in \mathcal{S}$, if q is a deadlock then q' weakly diverges.
- there exists $<\subseteq Q \times Q$ well-founded such that, for all $(q, q') \in \mathcal{S}$ and $(q, l, p) \in T$, if (q, l, q') is elided by $\mathcal{S}$ then $q > p$ or q' weakly diverges.

Theorem 5. *A weak simulation $\mathcal{S}$ is maximal if and only if it is progressive.*

Proof. Suppose that $\mathcal{S}$ is maximal. Let $(q, q') \in \mathcal{S}$ such that q is a deadlock then q weakly diverges so, by maximality, q' weakly diverges. Define $q > p$ if